Recording Acceleration Using Mouse Movement

Dr. Salwa Osama

WHAT IS THE ACCELERATION?

Speed

<u>Speed</u> is the rate of motion, or the rate of change of position.

It is expressed as distance moved (d) per unit of time(t).

Speed is a scalar quantity with dimensions distance/time.

Speed is measured in the same physical units of measurement as <u>velocity</u>, but does not contain an element of direction.

Speed is thus the magnitude component of velocity.

Velocity contains both the magnitude and direction components.

Common speeds of moving objects

FOR HUMAN BEINGS, AN AVERAGE WALKING SPEED IS ABOUT 3 MPH (~5 KM/H, 1.39M/S),

CAR CAN AVERAGE 65 MPH (~104 KM/H, 28.9 M/S) ON HIGHWAY

A 747 AIRPLANE HAS AN AVERAGE SPEED 565 MI/HR

Acceleration

$$\frac{\Delta v}{\Delta t} = a$$

- Acceleration (**symbol: a**) is the rate of change of velocity over time.It is a vector quantity, meaning it has both magnitude and direction.
- Any time the speed OR direction of an object changes, the object is accelerating.
- SI Unit: **m/s^2** (meters per second squared)
- Meaning: how much the velocity changes every second.

Acceleration

$$\frac{\Delta v}{\Delta t} = a$$

- If velocity increases → **positive acceleration**
- If velocity decreases → **negative acceleration (deceleration)**
- If direction changes but speed stays the same → **acceleration still occurs**
- Pressing the gas pedal → speed increases → positive acceleration
- Pressing the brake → speed decreases → negative acceleration
- Turning the car while keeping the same speed → direction changes → acceleration

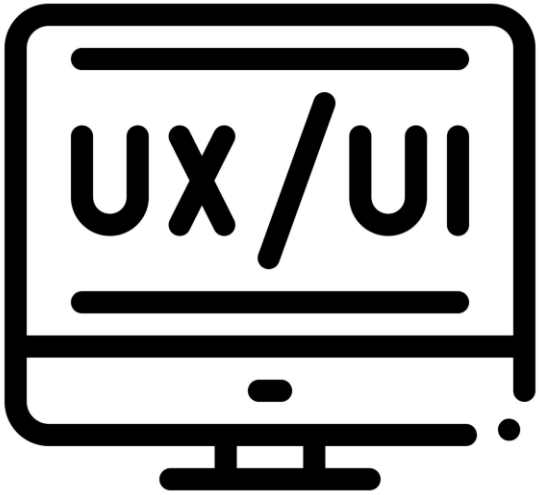
Mouse Movement Tracking

- Mouse tracking captures continuous x,y positions.
- Each movement event includes coordinates and a timestamp.
- Useful for studying motion digitally.
- Velocity is computed by change in position divided by change in time.
- Mouse data provides the needed time-stamped coordinates.
- Acceleration is computed by change in velocity over time.

Applications

UX & UI Design Improvement

- Mouse-movement data helps designers understand:
  - Which parts of the interface users look at first.
  - Where users hesitate, struggle, or get confused.
  - Whether buttons, menus, or forms are easy to find.
- This allows designers to **improve layouts, reduce confusion, and increase usability**.



Applications

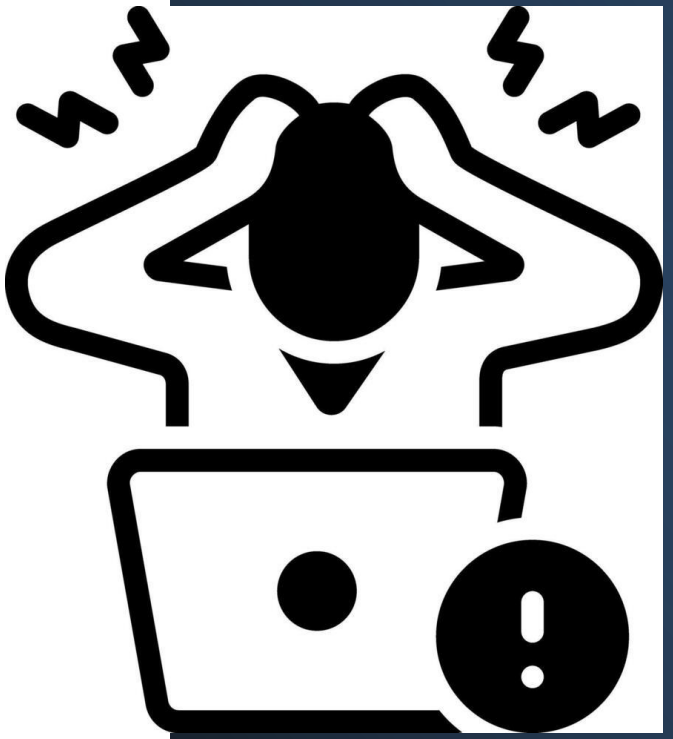
Website Performance & Conversion Optimization

- Companies use mouse-movement data to study:
  - How customers behave before clicking “Buy”.
  - Why users leave a page before completing a form.
  - Which areas attract attention or get ignored.
- This helps increase **sales, sign-ups, and user engagement**.

Applications

Detecting Frustration or Difficulty

- Mouse patterns can reveal when users are struggling:
  - Fast, shaky, or repeated movements → frustration.
  - Very slow movement → uncertainty.
  - Circling around buttons → difficulty finding the next step.
- This helps identify **problem areas** in software or websites.



Applications

Gaming Analytics

Mouse acceleration is extremely important in gaming:

- Measures player reaction time.
- Tracks aiming precision.
- Helps optimize mouse sensitivity settings.

- Coaches and players use this data to improve **performance and accuracy**.

Applications

Security & Fraud Detection

- Mouse movement patterns are **unique to each person**. Systems can detect:
  - Bots or automated scripts.
  - Fake users.
  - Fraudulent login attempts.
- It creates a type of **behavioral fingerprint**.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to S
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Import libraries

time → To measure timestamps and calculate time differences.

pyautogui → To get the current mouse position (x and y).

pandas → To store the data in a structured way and save it as a CSV file.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Initialize variables

- data=[] → List to store all the recorded data.
- prevTime=time.time() → Current time as starting reference.
- time.sleep(0.08) → Wait 0.08 seconds to ensure a small interval between measurements.
- prevX, prevY → Store the previous mouse coordinates.
- prevVX, prevVY → Initialize previous velocities in X and Y as 0.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Start tracking

- Print a message so the user knows tracking has started.
- try: → So that the user can stop the loop with Ctrl + C.
- while True: → Infinite loop to continuously track mouse movement.
- currentTime → Get the current time.
- dt → Calculate the time difference from the previous measurement.
- currX, currY → Get current mouse coordinates.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Calculate velocity

- Velocity in X: change in X position divided by elapsed time.
- Velocity in Y: change in Y position divided by elapsed time.
- This gives instantaneous velocity along each axis.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Calculate acceleration

Acceleration in X: change in X velocity divided by elapsed time.
Acceleration in Y: change in Y velocity divided by elapsed time.
az=0 → Mouse has no Z-axis, so acceleration along Z is 0.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Store data

Append the **current timestamp** and accelerations in X, Y, Z to the data list.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Update previous values

Update previous time, position, and velocity for the next iteration.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Sleep to control sample rate

Wait 0.08 seconds before the next measurement to control **sampling frequency**.
This prevents overloading the CPU and keeps consistent time intervals.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Handle user interruption

When the user presses **Ctrl + C**, stop the loop safely and print a message.

Coding

```python
import time
import pyautogui
import pandas as pd
data=[]
prevTime=time.time()
time.sleep(0.08)
prevX, prevY=pyautogui.position()
prevVX,prevVY=0,0
print("Tracking mouse Movment >>> Ctr + C to stop")
try:
    while True:
        currentTime=time.time()
        dt=currentTime-prevTime
        currX, currY=pyautogui.position()
        vx=(currX-prevX)/dt
        vy=(currY-prevY)/dt
        ax=(vx-prevVX)/dt
        ay=(vy-prevVY)/dt
        az=0
        data.append([currentTime, ax, ay, az])

        prevTime=currentTime
        prevX, prevY=currX, currY
        prevVX, prevVY=vx, vy

        time.sleep(0.08)
except KeyboardInterrupt:
    print("Stopp!")
df=pd.DataFrame(data,columns=["TimeStamp", "ax", "ay", "az"])
df.to_csv("mouseAcceleration.csv", index=False)
print("data are saved")
```

Save data

- Convert the data list into a **DataFrame** for easier handling.
- Specify column names: timestamp, ax, ay, az.
- Save the data as a CSV file named "mouseAcceleration.csv".
- Print confirmation that the data has been saved.

Thanks